DevOps Notes

1. Why DevOps?

Problem Before DevOps

Earlier, Development (Dev) and Operations (Ops) teams worked separately, often in silos.

- Developers → write code
- Operations → deploy & maintain code

Issues:

- Slow releases
- Frequent failures
- Blame game (Dev vs Ops)
- Poor communication
- Manual deployments

Why DevOps Was Needed

DevOps was introduced to:

- Deliver software faster
- Improve collaboration
- Reduce errors
- Automate processes
- Increase customer satisfaction

Goal: Faster, reliable, and continuous delivery of software.

2. What is DevOps?

Definition

DevOps is a culture and set of practices that combines Development (Dev) and Operations (Ops) to automate and improve the software delivery process.

In Simple Words:

DevOps means building, testing, deploying, and monitoring software continuously and automatically.

Key Focus Areas:

- Automation
- Collaboration
- Continuous delivery
- Monitoring & feedback

3. How DevOps Works?

DevOps follows a continuous lifecycle:

DevOps Lifecycle

Step	Description	Example Tools
1️⃣ Plan	Requirement gathering	Jira, Trello
2️⃣ Develop	Coding	Git, GitHub, GitLab
3️⃣ Build	Convert code into executable	Maven, Gradle
4️⃣ Test	Automated testing	Selenium, JUnit
5️⃣ Release	Version control & approvals	Jenkins
6️⃣ Deploy	Automatic deployment	Docker, Kubernetes, Ansible
7️⃣ Operate	Infrastructure management	AWS, Azure, Linux
8️⃣ Monitor	Performance & logs	Prometheus, Grafana, ELK

Feedback from monitoring goes back to Plan stage.

4. DevOps Benefits

Business Benefits

- Faster time-to-market
- Better customer experience
- Reduced downtime

Technical Benefits

- Automated deployments
- Faster bug fixes
- Consistent environments

Team Benefits

- Better collaboration
- Less conflict
- Shared responsibility

Overall Advantages

- ✔ Continuous Integration & Deployment (CI/CD)
- ✔ High availability
- ✔ Scalability
- ✔ Improved security

5. DevOps Principles

Core DevOps Principles (CALMS)

1. Culture

- Collaboration between Dev & Ops
- Shared responsibility

2. Automation

- Automate builds, tests, deployments
- Reduce human errors

3. Lean

- Deliver small changes quickly
- Remove waste

4. Measurement

- Monitor performance
- Track failures & recovery time

5. Sharing

- Share knowledge
- Share tools & feedback